Travel Planner
Project Plan/Project Management Plan

Department of Information Technology and Management

Victoria Li, Joyce Erika Reginaldo, Regina Maldonado Gutierrez

April 2026

1. Introduction

This PMP describes the project management processes that we will follow during execution of the Travel Planner project. The project's processes will align with plans and processes of the Project Management Accountability System (PMAS) Guide. New processes will be defined for any management areas not covered by the PMAS Guide. This PMP will govern the management practices across the life of the project. As those practices evolve, this document will be updated to reflect the changes.

1.1. Project Overview

The project's objective is to design and develop an intelligent travel planning application that integrates various APIs to assist users in making safe and informed travel decisions. The application will allow users to input a destination, and the app pulls real-time data from a weather API, currency exchange API, crime index API, and travel advisory API. A travel safety score will be computed based on aggregated risk factors & personalized travel packing suggestions may also be generated. This application will be programmed in Python, using Flask framework for the backend, as well as free tiers on the OpenMeteo API, Travel Advisory API, ExchangeRate API, and OpenMeteo GeocodingCities API. Major work activities include defining the data flow & scoring logic, UI/UX integration, backend/frontend development, unit testing for API calls, data analysis, and documentation.

1.2. Scope Statements

1.2.1. In-Scope Features

- User input interface for destination selection (city or country)
- Integration with external APIs
- Weather data retrieval
- Currency exchange rates
- Crime index data
- Travel advisory information
- Data processing and normalization from multiple APIs
- Travel Safety Score calculation based on defined criteria
- Travel recommendation engine (e.g., safe areas, best time to visit, budget suggestions)
- Display of results in a user-friendly format
- Basic data visualization (charts or summaries)
- GitHub repository for version control and collaboration
- Documentation and reporting (project plans, analysis, and results)

1.2.2. Out-of-Scope Features

- Real-time booking systems (flights, hotels, rentals)
- User account creation or authentication systems
- Mobile app development (unless optionally added later)
- Advanced AI/ML predictive modeling beyond basic scoring
- Payment processing or financial transactions
- Integration with private or paid enterprise-level APIs (beyond free tiers)

1.3. Timeline & Milestones

Tasks	Start	End	Owner	Deliverable
Define roles & project idea	Week 1	Week 1	All	This document
Research APIs/data sources	Week 1	Week 1	All	X
Create GitHub repo & documents	Week 1	Week 1	Application Developer	GitHub repo
Design system architecture & UI	Week 2	Week 2	Application Dev	X
Backend/frontend development (API integration & result display)	Week 2	Week 2	Application Dev	Python code in GitHub
Perform unit/integration testing for API calls	Week 3	Week 3	All	Test Document
Analyze collected data trends & generate visualizations	Week 3	Week 3	Data Analyst	Data Analysis Document
Write technical & user documentation	Week 3	Week 3	Documentation Writer	README
Complete Risk Management Plan	Week 3	Week 3	Documentation Writer	Risk Management sheet
Complete Earned Value Sheet	Week 3	Week 3	Documentation Writer	EV sheet

2. Project Organization

2.1. Roles & Responsibilities

- Project Manager -> tracks team progress & updates plan
- Application Developer -> develop application & handle backend logic
- Documentation Writer -> document progress & tests
- Data Analyst -> perform dataset & gain insights

3. Workflow Guidelines

3.1. Git Workflow

Development process:

1. Pull latest code from main
2. Create feature branch
3. Commit changes with clear messages
4. Push branch to GitHub
5. Create a pull request

6. Merge

3.2. API Integration Workflow

1. Research API endpoints & limits
2. Create dedicated files for each API
    a. weather_api.py
    b. advisory_api.py
    c. city_api.py
    d. currency_api.py
3. Standardize outputs into a common format
4. Handle errors & rate limits
5. Write unit tests for each API module

3.3. Testing Workflow

1. Log issue in chat
2. Fix in dedicated branch
3. Retest before merging

4. Project Success Criteria

The project will be considered successful if:

- The application successfully retrieves and integrates all required data sources
- A functional and logical Travel Safety Score is generated
- Users can input destinations and receive meaningful recommendations
- The system operates without critical errors
- All required documentation and deliverables are completed
- The project is fully maintained and version-controlled on GitHub

5. Communication Management Plan

5.1. Communication Strategies/Tools

- Microsoft Teams (daily text updates/progress checks)
- Microsoft Word (documentation)
- Microsoft Excel (Earned Value & Risk Management)
- GitHub (version control)

6. Risk Management Plan

Risk identification & mitigation strategies found in this spreadsheet:

Risk_Management_Log_Template.xls

7. Software Configuration Management (SCM) Plan

7.1. API Implementation

1) Weather API Implementation
    a) In weather_api.py, calls OpenMeteo to return daily temps and averages when inputting latitude and longitude
    b) Handle errors for invalid coordinates, API failure, missing data
2) Travel Advisory API Implementation
    a) In advisory_api.py, calls Travel Advisory API to return advisory score and message when inputting country code
    b) Handle errors for invalid country, missing advisory, API failure
3) GeoDB API Implementation
    a) In city_api.py, calls OpenMeteo GeocodingAPI to return latitude, longitude, country code, cleaned city name when inputting city name
    b) Handle errors for city not found, multiple matches, API failure
4) ExchangeRate API Implementation
    a) In currency_api.py, calls ExchangeRate API to return exchange rate for destination currency using base currency as input
    b) Handle errors for missing currency, API failure

7.2. Services Logic

1) In intelligence.py, create a travel score function, classify risks, and generate suggestions for packages
    a) travel score function will input advisory score, avg temp, currency stability and output numeric score
    b) classify risk function will input travel score and output levels of risk (Low to High)
    c) package suggestion function will input temperature and risk level and output list of recommendations
2) In charts.py, generate temperature chart using matplotlib w/ labels, title, and grid

7.3. GUI + Integration

1) Build website/app w HTML
    a. Entry box for city
    b. “Check travel info” button
    c. “Show weather info” button
    d. Output text area
2) Integrate APIs using Flask
    a. Get city info
    b. Get weather
    c. Get advisory
    d. Get currency
    e. Compute travel score
    f. Classify risk
    g. Generate packing list
    h. Return JSON
3) Display results
    a. Show city+country
    b. Safety score

    c. Risk level
    d. Advisory message
    e. Average temperature
    f. Currency rate
    g. Packing list
4) Add chart button
    a. Calls charts.py temperature plot function

Version control & GitHub repo:
https://github.com/victorialix/OSINT-Project

8. Earned Value Sheet

EV sheet can be found here:
https://github.com/victorialix/OSINT-Project/blob/main/docs/earned_value_sheet.md

9. Test Documentation

Test documentation can be found here:
https://github.com/victorialix/OSINT-Project/blob/main/docs/test_document.md

10. Data Analysis

https://github.com/victorialix/OSINT-Project/blob/main/docs/data_analysis.md